Real-Time HDMI Threshold and Motion Detection IP on PYNQ-Z2

HDMI In --> **Custom HLS IP** --> Threshold HDMI Out --> Motion summary --> GPIO

↓

PS

↓

UART

Sora Kakigi • Thinh Nguyen • Yi Chieh Huang • Justin Lu

Why this project?

Real-time FPGA video processing is a strong fit for HLS and a clear live demo.

HDMI live streaming provides an immediate, visual hardware demo.

Motion detection is intuitive: compare the current frame with previous-frame state.

FPGA acceleration is attractive because the work repeats for every pixel.

PYNQ-Z2 gives a practical platform for end-to-end HLS → Vivado → board integration.

Goal

Real-time HDMI thresholded display + previous-frame motion reporting

Original target vs. final delivered system

Original target

Compare the current frame with the previous frame.

Generate a motion mask directly in hardware.

Display the motion result directly on HDMI output.

Final delivered system

Stable thresholded HDMI output.

Background previous-frame motion analysis.

Motion count + 4×4 region reporting through UART.

Readable index grid and binary grid on the PS side.

Final demo in one slide

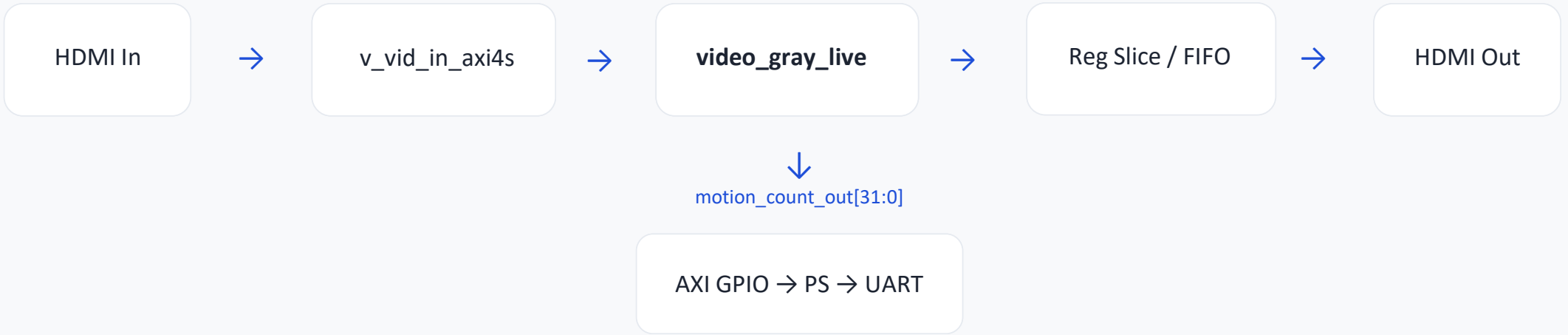
HDMI output:
stable thresholded image

UART output:
motion_count
4×4 index grid
4×4 binary grid

```
[HB] app alive=12, rx_lock=1,
tx_lock=1, motion_count=428, motion=1

Index grid                | Binary grid
 0  2  0  0                | 0 1 0 0
 0  0  7  0                | 0 0 1 0
 0  0 11 12                | 0 0 1 1
 0  0  0 16                | 0 0 0 1
```

System architecture



Inside the HLS IP

Display path

Read incoming RGB pixel.

Use the green channel as a grayscale proxy.

Apply threshold.

Output a stable black/white HDMI image.

Background motion path

Sample one point per 4×4 block.

Compare current binary value with previous-frame stored value.

Increment motion_count when the block changes.

Set the bit for the active 4×4 region.

Motion detection method

Frame size: 1280 × 720.

Motion grid after 4×4 downsampling: 320 × 180.

One motion sample is computed for each 4×4 block.

Binary motion logic: curr_bin = (g ≥ T), motion = curr_bin XOR prev_bin.

```
curr_bin = (g >= T)

motion = curr_bin XOR
prev_bin

if (motion):
  motion_count++
  region_mask |= (1 <<
region_idx)
```

4×4 region mapping

```
 1   2   3   4
 5   6   7   8
 9  10  11  12
13  14  15  16
```

The region mask uses 16 bits.

The PS prints both an index grid and a binary grid for readability.

Major technical challenge

Threshold-only HDMI output was stable.

Previous-frame read/write in the background was possible.

But when HDMI output directly depended on previous-frame comparison, TX lock often failed.

The problem was not motion detection itself, but preserving a stable live HDMI output path.

Debugging process

1. HDMI passthrough bring-up
2. Grayscale output
3. Threshold output
4. Previous-frame access in background
5. Direct motion-mask output attempt
6. TX lock failure
7. Added register slice / FIFO
8. Re-tested several HLS versions
9. Final separation of display path and motion-report path

What finally worked

Version / behavior	Result
Pass-through	Stable
Threshold only	Stable
Threshold + background memory	Stable
Direct motion-mask output	Unstable
Final design	Stable

PS(Processing System) and UART reporting

The PS configures the clock wizard and VTC.

The PS monitors HDMI RX/TX lock signals.

The PS reads motion_count_out through AXI GPIO.

The PS prints motion_count plus both 4×4 visualizations.

```
Index grid       | Binary grid
 0  2  0  0      | 0 1 0 0
 0  0  7  0      | 0 0 1 0
 0  0 11 12      | 0 0 1 1
 0  0  0 16      | 0 0 0 1
```

Final result

HDMI threshold output works.

RX lock = 1 and TX lock = 1.

Motion count changes with scene changes.

The 4×4 region report works through UART.

The final system is a reliable motion-reporting demo on PYNQ-Z2.

Demo status

HDMI stable
UART reporting
4×4 regions working

Lessons learned

- Algorithm correctness alone is not enough in FPGA video systems.
- Live HDMI output paths are extremely sensitive to timing and synchronization.
- Previous-frame logic is much easier to support in background analysis than in direct display output.
- Architectural partitioning was the key to a stable final design.

Future work

- Frame-buffer based motion-mask overlay using DDR / VDMA.
- Region-wise motion thresholds and temporal smoothing.
- Bounding boxes for moving regions.
- Adaptive thresholding and more detailed spatial reporting.
- Object-level tracking on top of the current pipeline.